

Relatório Técnico — Tech Challenge

Fase 3

Assistente Virtual Médico com Fine-Tuning de LLM e Orquestração LangChain/LangGraph

Curso: Pós-graduação FIAP — IA para Devs

Fase: 3

Modelo publicado: [fiap-hospital-helper/hospital-helper-qwen2.5-1.5b](#) (tag v2.0)

Modelo base: Qwen/Qwen2.5-1.5B-Instruct

1. Introdução

Este relatório atende aos requisitos obrigatórios do edital da Fase 3 do Tech Challenge, que solicita a criação de um assistente virtual médico treinado com dados próprios do hospital, capaz de auxiliar condutas clínicas, responder dúvidas de médicos e sugerir procedimentos com base em protocolos internos, além de orquestrar fluxos de decisão automatizados e seguros via LangChain/LangGraph.

O projeto foi dividido em quatro entregas técnicas exigidas pelo edital:

1. Fine-tuning de LLM com dados médicos internos;
2. Criação de assistente médico com LangChain;
3. Segurança e validação;
4. Organização do código.

Cada uma é detalhada a seguir, seguida da avaliação formal do modelo e da análise dos resultados.

2. Entrega 1 — Fine-tuning de LLM com dados médicos internos

2.1 Escolha do modelo base

Foi adotado o **Qwen2.5-1.5B-Instruct** (Alibaba Cloud, licença Apache 2.0) como modelo base (ADR-001). A escolha se justificou por:

- caber em GPU T4 (15GB) do Google Colab, mesmo sem quantização agressiva;
- suporte multilíngue nativo (português e inglês);
- ser *instruction-tuned*, favorecendo o SFT (*Supervised Fine-Tuning*) subsequente;
- licenciamento aberto para uso acadêmico e comercial.

Alternativas como LLaMA 3.2 1B, Mistral 7B, GPT-2 e BioMedLM foram descartadas por limitações de suporte a português, tamanho incompatível com o hardware disponível ou desempenho insuficiente em Q&A estruturado.

2.2 Dados de treinamento e curadoria

Os dados de fine-tuning combinam:

- **PubMedQA** — Q&A clínico baseado em publicações do PubMed;
- **MedQuAD** — Q&A de saúde derivado de fontes do NIH;
- Protocolos clínicos internos **FHEMIG** e **PCDT**, extraídos de PDFs públicos;
- Laudos médicos estruturados e anonimizados, usados como base de conhecimento complementar (RAG).

Todos os registros passam por **curadoria automática versionada** (curation-v1, ADR-018) antes de entrarem no conjunto de treino: pergunta e resposta não vazias, cumprimentos mínimos, estrutura válida (MedQuAD) e extração de texto bem-sucedida (protocolos). O pipeline gera `curation_report.json`, persistido também no MongoDB, com contagens de aceitos/rejeitados e motivos por fonte, garantindo rastreabilidade total da preparação dos dados — atendendo à exigência do edital de "preprocessing, anonimização e curadoria".

Os laudos médicos usados na RAG são anonimizados antes da indexação (ADR-015): nome do paciente e médico solicitante são mascarados, e campos identificadores (`id_laudo`, CPF, CRM) nunca são enviados ao conteúdo ou aos metadados indexados, em conformidade com a LGPD.

O conjunto final de treino somou **17.407 exemplos** no formato de instrução:

```
### Instrucao:
```

```
Responda em pt-BR usando o contexto clinico fornecido.
```

```
### Entrada:
Pergunta: {pergunta}
Contexto:
{contexto}
```

```
### Resposta:
{resposta}
```

2.3 Técnica de fine-tuning

Foi utilizado **LoRA/PEFT** (ADR-002) com quantização 4-bit (**QLoRA**, ADR-003) para viabilizar o treinamento em GPU T4 do Google Colab (ADR-006):

```
LoraConfig(
    r=16,
    lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
)

BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

Essa combinação reduz os parâmetros treináveis a ~0,5–2% do total (contra 100% em um full fine-tuning), tornando viável treinar em ~1–2h em GPU gratuita, com baixo risco de *catastrophic forgetting* e preservação do conhecimento geral do modelo base.

O treinamento foi conduzido via `trl.SFTTrainer/SFTConfig`, com checkpoints periódicos (`save_strategy="steps"`) e suporte a retomada automática (`resume_from_checkpoint`) para lidar com os limites de sessão do Colab gratuito. Ao final, os adaptadores LoRA foram mesclados ao modelo base (`merge_and_unload`) e publicados no Hugging Face Hub sob a organização `fiap-hospital-helper`, com tag oficial `v2.0`.

2.4 Detecção automática de hardware

O backend do projeto implementa detecção automática de GPU/CPU (ADR-009), aplicando `bfloat16/float16` em GPU e `float32` em CPU, sem exigir configuração manual — o que também orientou a estratégia de fallback CPU do QLoRA.

3. Entrega 2 — Criação do assistente médico com LangChain/LangGraph

3.1 Arquitetura do agente

O assistente médico é um serviço isolado (porta 8001), orquestrado como um **grafo StateGraph do LangGraph** (ADR-011), compilado com 8 nós e persistência de estado via MongoDBSaver (ADR-017).

O LangGraph foi preferido a LLMChain simples ou a *Agents* (ReAct) por permitir **roteamento condicional explícito com early-exit**: quando o validador de tópico ou o guardrail de segurança detecta uma violação, o grafo pula diretamente para o `audit_logger`, sem invocar RAG nem LLM — reduzindo latência, custo e risco de vazamento de informação.

3.2 Jornadas de uso

O agente cobre duas jornadas descritas no documento de referência do produto:

Jornada 1 — Dúvida clínica pontual (Q&A com protocolos internos).

O médico envia uma pergunta sem vincular a um paciente. O fluxo recupera contexto via RAG sobre os protocolos internos (FHEMIG/PCDT/PubMedQA/MedQuAD), a LLM fine-tunada gera a resposta combinando pergunta e contexto, a resposta cita a fonte (explainability), reforça a necessidade de validação humana e é registrada em log de auditoria.

Jornada 2 — Consulta contextualizada por paciente.

Ativada via campo explícito `patient_name` na API (POST `/agent/chat`), evitando heurísticas frágeis de extração de nome por regex/NER. O nó `patient_context_retriever` consulta GET `/medical-record` no backend, extrai apenas os campos clinicamente relevantes (avaliação/diagnóstico, prescrição, alergias, sinais vitais) e descarta PII (CPF, leito, data de nascimento). O nó `rag_retriever` enriquece a busca vetorial combinando o diagnóstico do paciente com a pergunta. O nó `context_summarizer` calcula a estimativa de tokens do pacote completo e, caso ultrapasse a janela SFT de 3.000 tokens do modelo fine-tunado, aciona a Groq (compound-mini, 70k TPM) para condensar o contexto em menos de 800 tokens, com fallback determinístico em Python puro caso a API esteja indisponível (ADR-020). O frontend expõe badges (📄 Prontuário consultado, ⚡ Contexto resumido) com detalhamento auditável dos dados usados.

Jornada 3 — Fluxo automatizado de decisão.

Cenário citado no desafio: a partir de um evento (admissão, novo exame), o LangGraph aciona etapas de verificação de exames pendentes, sugestão de tratamento e emissão de alerta à equipe, mantendo a decisão final sempre humana e o registro de justificativa para rastreabilidade. A arquitetura de nós já está preparada para esse acionamento assíncrono; a

implementação completa está prevista para fases posteriores do produto.

3.3 Inferência da LLM em produção

Como o modelo fine-tunado exige GPU, foi adotada uma **arquitetura híbrida de inferência** (ADR-012):

- **Desenvolvimento:** notebook Colab expõe um servidor FastAPI (POST /generate) via túnel **ngrok**, usado durante iteração e testes (ADR-010);
- **Produção/entrega:** o modelo é servido em **HuggingFace Spaces com ZeroGPU** (app.py, decorator `@spaces.GPU`), com URL pública e persistente, alocando GPU sob demanda apenas durante a inferência.

O módulo `llm_client.py` do agente detecta automaticamente o provedor ativo (GradioSpaceLLMClient ou FastApiLLMClient) a partir da URL configurada, expondo uma interface única `generate(pergunta, contexto, prompt)` ao restante do pipeline.

4. Entrega 3 — Segurança e validação

O edital exige limites de atuação do assistente (nunca prescrever diretamente), logging detalhado e explainability. Essas exigências foram implementadas como uma **camada de segurança determinística, desacoplada da LLM** (ADR-013):

Camada	Mecanismo
1. Validação de tópico (topic_validator)	Heurística de keywords médicas (PT/EN) + regex contextual; rejeita perguntas fora do domínio de saúde antes de qualquer chamada externa
2. Guardrails de segurança (safety_guard)	Lista negra via regex: bloqueia prescrição com dose específica, diagnóstico definitivo e substituição do médico
3. Template SFT preservado (llm_generator)	O prompt enviado à LLM segue estritamente o template de treino (Instrução/Entrada/Resposta), sem <i>system prompts</i> extensos que degradariam a fidelidade dos pesos LoRA
4. Response formatter (response_formatter)	Injeta disclaimer legal obrigatório, remove artefatos e loops de repetição, garante requires_human_validation: true invariante
5. Auditoria (audit_logger)	Persiste toda interação — inclusive as bloqueadas — em agent_audit_logs no MongoDB

Essa arquitetura foi deliberadamente escolhida em vez de delegar a segurança a instruções em linguagem natural para a LLM, que seriam vulneráveis a *jailbreak* e *prompt injection*, especialmente relevante em um modelo compacto (1.5B) sensível à estrutura de tokens de treino.

Explainability

Toda resposta cita a fonte da informação de duas formas complementares:

1. **Citação inline** no corpo da resposta (ex.: [Fonte: PubMedQA/MedQuAD, score: 0.87]);
2. **Campo estruturado** sources na resposta da API, com dataset, tipo de fonte e score de similaridade.

Na Jornada 2, os campos patient_context_used e patient_fields_used documentam exatamente quais seções do prontuário alimentaram a resposta.

Logging e auditoria

A coleção agent_audit_logs no MongoDB registra, por interação: session_id, query, topic_valid, safety_triggered/safety_reason, documentos RAG usados, resposta bruta e final, fontes citadas, presença de disclaimer, duração, e — na Jornada 2 — uso e campos do prontuário e modo de sumarização de contexto. O grafo mantém memória de sessão persistente via MongoDBSaver (ADR-017), separada da auditoria, permitindo continuidade conversacional entre chamadas.

5. Entrega 4 — Organização do código

O projeto é modularizado em três serviços independentes, documentados individualmente com README próprio:

Serviço	Tecnologia	Responsabilidade
Frontend	React + TypeScript + Vite	UI para pré-processamento, RAG e chat com o assistente
Backend	Python + FastAPI	Pré-processamento de datasets, geração/consulta RAG, consulta de laudos e prontuários
Agente Médico	Python + FastAPI + LangGraph	Orquestração do assistente com guardrails e auditoria
Database	MongoDB	Estado de execuções, checkpoints de sessão e logs de auditoria

A orquestração local é feita via **Docker Compose** (ADR-008), com suporte a GPU Nvidia (runtime: nvidia) e fallback CPU (docker-compose.cpu.yaml), subindo toda a stack com um único comando. O processamento pesado (download, extração, tradução) roda via **FastAPI BackgroundTasks** (ADR-005), com polling de status pelo frontend.

As 20 decisões arquiteturais do projeto estão documentadas como **ADRs** no formato MADR, cobrindo desde a escolha do modelo base até a compatibilização de bibliotecas de embeddings, garantindo rastreabilidade das escolhas técnicas ao longo do desenvolvimento.

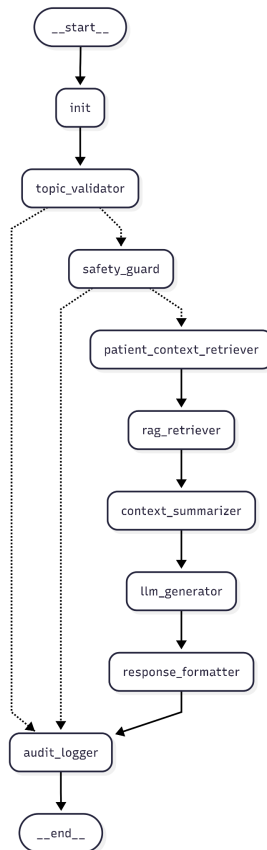
Documentação da Arquitetura: A documentação completa da arquitetura do projeto encontra-se na pasta /docs do repositório oficial, incluindo os documentos técnicos e diagramas que detalham as decisões e componentes da solução:

<https://github.com/vagner-nascimento/fiap-pos-ia-para-devs-fase3-tech-challenge/tree/main/docs>

6. Diagrama do fluxo LangChain/LangGraph (visão consolidada)

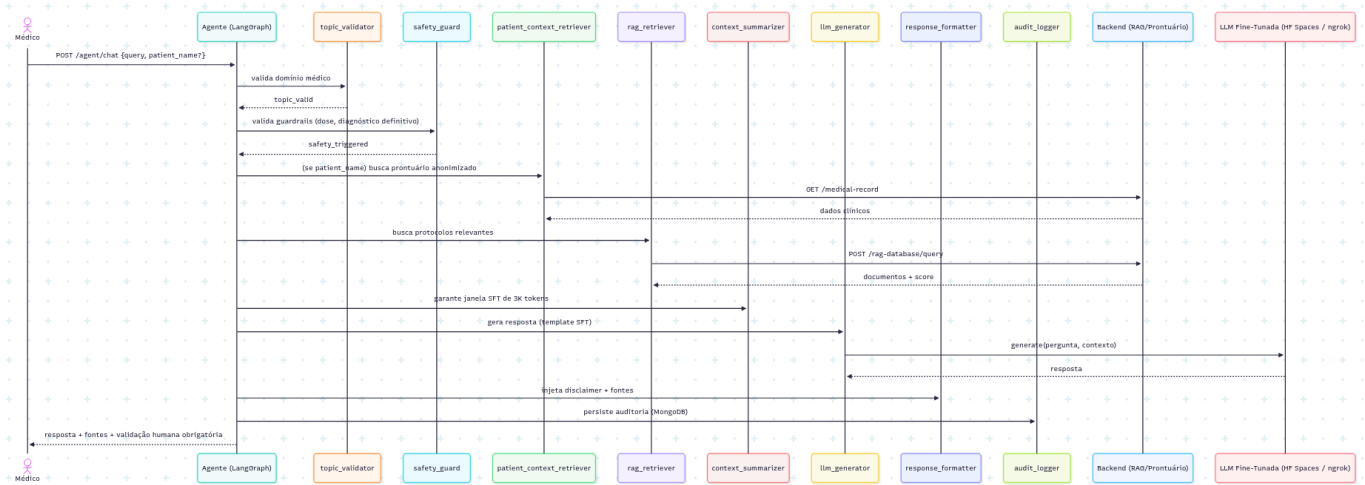
6.1 Grafo compilado do agente (LangGraph)

Abaixo está a representação visual do grafo efetivamente compilado (StateGraph) que orquestra o agente médico, com os nós de decisão condicional (early-exit) explícitos:



As rotas condicionais representam a lógica de early-exit: se `topic_validator` rejeita a pergunta (fora do domínio médico) ou `safety_guard` detecta uma violação (prescrição com dose, diagnóstico definitivo etc.), o grafo pula diretamente para `audit_logger` sem passar pelo restante do pipeline — evitando chamadas desnecessárias ao RAG e à LLM. Quando ambas as validações passam, o fluxo segue a rota principal por `patient_context_retriever` → `rag_retriever` → `context_summarizer` → `llm_generator` → `response_formatter`, encerrando sempre em `audit_logger` antes do encerramento, garantindo que **toda** interação — validada, bloqueada ou completa — seja auditada.

6.2 Fluxo de execução (sequência)



7. Avaliação do modelo e análise dos resultados

A avaliação quantitativa completa está documentada em docs/avaliacao-modelo.md e na metodologia do ADR-016. Resumo dos principais achados:

7.1 Metodologia

- **Dataset golden test:** 50 casos clínicos balanceados em pt-BR (golden_test_qa.json) — protocolos de emergência (dor torácica, sepse, cetoacidose, AVC, anafilaxia, TEP, PCR, guardrail ético) e casos especializados (cardiologia, infectologia, pneumologia, cirurgia, pediatria, farmacologia);
- **Ambiente:** Google Colab, GPU Tesla T4, precisão bfloat16;
- **Métricas:** ROUGE-1/2/L, BLEU-4, latência média, throughput;
- **Comparação:** modelo fine-tunado (v2.0) vs. modelo base (Qwen2.5-1.5B-Instruct), sob três configurações de decodificação (Greedy, Amostragem suave, Janela ampla).

7.2 Resultados consolidados

Configuração	ROUGE-L (Δ)	BLEU-4 (Δ)	Latência (Δ)
A — Greedy (300 tokens)	+8,9%	-27,0%	-44,5%
B — Amostragem (300 tokens)	+5,9%	-12,0%	-60,7%
C — Janela ampla (450 tokens)	+15,0%	+23,3%	-57,6%

A **Configuração C** (max_new_tokens=450, temperature=0.10, top_p=0.85, repetition_penalty=1.10) foi adotada como padrão de produção por superar o modelo base em ROUGE-1 (+3,5%), ROUGE-L (+15,0%) e BLEU-4 (+23,3%), com latência 57,6% menor (5,58s vs. 13,15s).

7.3 Achados qualitativos

A análise lado a lado de casos reais mostrou que o modelo base comete alucinações terminológicas graves (siglas inexistentes, condutas iatrogênicas) e recomendações incorretas

— por exemplo, indicando anti-histamínico intravenoso como primeira linha na anafilaxia, quando o correto é adrenalina intramuscular. O modelo fine-tunado corrigiu esses desvios na maioria dos casos avaliados, além de responder de forma mais objetiva e recusar corretamente pedidos de prescrição direta (caso de guardrail ético), exigindo avaliação médica presencial. O ganho mais consistente entre todas as configurações foi o **ROUGE-L** (estrutura sequencial das condutas), evidenciando que o fine-tuning ensinou o modelo a seguir a ordem sintática das recomendações de sociedades médicas e do Ministério da Saúde, além de reduzir drasticamente a prolixidade e os loops de repetição do modelo base.

7.4 Limitações identificadas

- O ganho em BLEU-4 nas Configurações A e B foi negativo, indicando que a métrica de n-gramas exatos é sensível à paráfrase clínica legítima;
- Em casos raros que exigem respostas acima de 450 tokens, há risco de truncamento (~2% dos casos de referência);
- O modelo de 1.5B parâmetros tem capacidade de raciocínio limitada frente a modelos maiores, mitigada pela camada de RAG e pelos guardrails determinísticos.

8. Conformidade com o edital

Requisito do edital	Onde foi atendido
Fine-tuning de LLM com dados médicos internos	Seção 2 — LoRA/QLoRA sobre Qwen2.5-1.5B, dados de protocolos, FAQs e laudos
Preprocessing, anonimização e curadoria	Seção 2.2 — curadoria versionada (curation-v1) e anonimização LGPD dos laudos
Assistente médico com LangChain (pipeline + RAG + contexto do paciente)	Seção 3 — LangGraph de 8 nós, Jornadas 1 e 2
Limites de atuação (nunca prescrever diretamente)	Seção 4 — guardrails determinísticos e disclaimer obrigatório
Logging detalhado e auditoria	Seção 4 — agent_audit_logs no MongoDB
Explainability (fonte da resposta)	Seção 4 — citação inline + campo sources
Organização modular do código	Seção 5 — três serviços + Docker Compose + ADRs
Avaliação do modelo e análise dos resultados	Seção 7

9. Conclusão

O modelo fine-tunado [fiap-hospital-helper/hospital-helper-qwen2.5-1.5b](#) (v2.0), combinado ao pipeline LangGraph do agente médico, atende integralmente aos requisitos obrigatórios da Fase 3 do Tech Challenge: fine-tuning viável em hardware acessível, assistente médico contextualizado por paciente e por protocolos internos, camadas de segurança determinísticas que impedem prescrição direta, explainability nas respostas e auditoria completa de cada interação. A avaliação empírica confirma ganhos de precisão protocolar, redução de alucinações e latência significativamente menor frente ao modelo base, validando a arquitetura adotada para uso em ambiente hospitalar assistido por profissional de saúde.